Modèles algorithmiques fondamentaux sur structures à accès direct

Thème principal : Initiation à l'algorithmique et patrons de conception (Design Patterns) sur structures indicées

Synthèse exécutive : Ce cours introduit les trois grands modèles algorithmiques indispensables à maîtriser en BUT 1 Informatique : le parcours complet avec traitement systématique, le parcours complet avec traitement conditionné, et la recherche de première occurrence. L'objectif est de fournir des schémas de solutions réutilisables et rigoureux, applicables principalement sur des structures à accès direct (tableaux, chaînes de caractères), en insistant sur l'analyse préalable de la collection, la gestion rigoureuse des bornes (indices de 0 à N-1 pour éviter le dépassement 'out of bounds'), le principe de pessimisme algorithmique et l'obligation d'une double condition d'arrêt pour toute recherche.

1. Le Pourquoi & Les Mécanismes d'Accès aux Structures de Données

Avant de concevoir le moindre algorithme, l'étape zéro de l'informaticien consiste à analyser le contrat d'interface et la nature de la collection de données manipulée. Le choix d'un modèle algorithmique dépend strictement des opérations permises par la structure. Si l'on travaille sur une structure à accès direct (tableaux, chaînes de caractères), on peut cibler n'importe quel indice immédiatement. En revanche, sur un flux séquentiel (fichier texte, flux CSV), il est impossible de sauter directement à un indice donné : il faut obligatoirement parcourir les éléments un à un depuis le début. Les modèles présentés dans ce cours s'appliquent exclusivement aux structures à accès direct. Tenter d'utiliser une notation par indice (ex: T[i]) sur un flux séquentiel constitue une erreur conceptuelle critique.

- Structure de données : Collection organisée d'éléments de même type (ex: tableaux, chaînes de caractères).
- Accès direct : Capacité d'atteindre instantanément n'importe quel élément via son indice ou sa clé en temps constant O(1), sans visiter les éléments précédents.
- Accès séquentiel : Contrainte imposant de se positionner au début puis de progresser pas à pas (début, suivant, fin) sans saut direct possible (ex: fichiers CSV bruts, listes chaînées simples).
- Accès dynamique : Structure hybride proposant simultanément des primitives d'accès séquentiel et d'indexation directe.

Cas concret enseignant : *Le professeur insiste sur un piège classique en partiel : tenter d'appliquer un modèle d'accès direct (comme T[i]) sur un fichier CSV brut ou un fichier texte. Comme le système d'exploitation ne fournit qu'un curseur séquentiel, un tel code est sémantiquement faux et conduit directement à un zéro sur la question.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une zone définie doit être modifié, affiché ou calculé de façon identique, sans aucune exclusion (ex: initialiser toutes les cases d'un tableau à zéro ou passer tous les caractères d'une chaîne en majuscules). Le nombre d'itérations étant connu avant d'entrer dans la boucle, on utilise systématiquement une boucle bornée 'Pour'. En notation standard, les indices d'un tableau de N éléments vont de 0 à N - 1. Le piège classique des étudiants consiste à faire boucler l'indice jusqu'à N, ce qui déclenche une exception critique de dépassement d'indice (out of bounds).

- Objectif : Appliquer un traitement uniforme et inconditionnel à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : Utilisation obligatoire d'une boucle bornée 'Pour' (car le nombre d'itérations est calculable à l'avance : Fin - Début + 1).
- Gestion stricte des bornes : Pour une structure de N éléments indicée à partir de 0, la boucle varie strictement de 0 à N - 1.
- Erreur critique (Out of Bounds) : Faire varier l'indice jusqu'à N provoque un dépassement de capacité mémoire.

Cas concret enseignant : *Exemple concret donné en amphi : 'Afficher un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente des débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat à l'exécution lors de la tentative de lecture de la case mémoire T[N] qui n'appartient pas au tableau.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

C'est une évolution directe du modèle précédent. Le parcours reste complet (on visite tous les éléments de la borne A à la borne B), mais l'action à l'intérieur de la boucle est conditionnée par un test logique. Par exemple, compter le nombre de voyelles dans une chaîne de caractères ou afficher uniquement les nombres positifs d'un tableau. La condition peut être simple (ex: supérieur à 0) ou complexe (cumulant plusieurs critères, voire impliquant les éléments voisins si l'algorithme est plus sophistiqué). Si la condition implique des voisins (ex: T[i] > T[i-1]), les bornes doivent être ajustées pour éviter tout déréférencement hors limites.

- Parcours exhaustif : Tous les éléments de la zone sont inspectés de la borne de début à la borne de fin.
- Traitement sélectif : L'action n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).
- Architecture : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.
- Conditions complexes : Le prédicat peut combiner plusieurs critères logiques (ET, OU) ou comparer l'élément courant avec ses voisins immédiats (i-1 ou i+1).

Cas concret enseignant : *L'analogie de la chaîne de montage en usine : 'Chaque pièce défile obligatoirement devant le contrôleur qualité (parcours complet systématique d'inspection), mais le contrôleur n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).'*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux parcours complets, une recherche de première occurrence s'appuie sur une itération non bornée ('Tant que') car le nombre d'itérations est imprévisible. L'informaticien doit adopter une posture pessimiste en postulant que l'élément n'existe pas. Un algorithme de recherche valide DOIT impérativement posséder deux conditions d'arrêt : (1) le succès de la recherche, (2) l'épuisement complet de la collection. Deux variantes structurelles existent : la variante 1 (avec rupture immédiate, idéale pour récupérer la position exacte) et la variante 2 (Tant que global, plus simple à structurer syntaxiquement).

- Objectif : Localiser le tout premier élément d'une collection vérifiant une propriété donnée et interrompre l'exploration immédiatement.
- Pessimisme algorithmique : Initialisation systématique d'un indicateur booléen 'trouvé' à Faux avant d'entamer l'exploration.
- Itération non bornée : Utilisation obligatoire d'une boucle 'Tant que' car le nombre de répétitions est inconnu a priori.
- Double condition d'arrêt obligatoire : Arrêt dès que l'élément est trouvé OU dès que la structure a été entièrement explorée sans succès.
- Variante 1 (Rupture immédiate) : Interruption nette dès la découverte ; conserve l'indice i exactement positionné sur l'élément trouvé.
- Variante 2 (Tant que global) : Test unique en tête ('Tant que i < N et NON trouvé') ; syntaxiquement plus standard mais avance d'un pas supplémentaire avant de sortir.

Cas concret enseignant : *Devise du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans votre collection, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle que tout algorithme de recherche présenté en examen avec une seule condition d'arrêt est sanctionné par une note éliminatoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, car une structure à accès dynamique propose à la fois des primitives d'accès séquentiel et d'accès direct. On peut donc exploiter ses mécanismes d'accès direct pour appliquer le modèle.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que la variante 1 (avec rupture) est indispensable si l'on a besoin de récupérer la position exacte (l'indice) de l'élément trouvé. La variante 2 (Tant que global), quant à elle, offre une syntaxe plus épurée et plus simple à coder lorsqu'on souhaite uniquement tester l'existence booléenne sans se soucier de l'indice exact.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut impérativement adapter les bornes de la boucle (par exemple, démarrer à l'indice 1 au lieu de 0 si l'on compare avec T[i-1]) pour éviter tout dépassement de capacité mémoire (out of bounds).

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser une recherche de première occurrence, mais un 'Parcours complet avec traitement conditionnel', puisque l'on doit inspecter l'intégralité de la structure et filtrer toutes les occurrences.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. On doit inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle canonique est donc le 'Parcours complet avec traitement conditionnel'. Étape 2 : Détermination des bornes. En indexation standard (commençant à 0), pour N éléments, les indices s'étendent de 0 à N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Structure de contrôle itérative. Nombre d'itérations connu -> Boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Filtre logique et action. À chaque tour, évaluer 'Si T[i] > 0 Alors' et exécuter 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit le parcours complet exhaustif sans risque de débordement d'indice (arrêt strictement à N-1) et filtre sélectivement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Parce qu'en indexation standard à partir de 0, la énième et dernière case d'un tableau de N éléments se trouve à l'indice N - 1. Tenter d'accéder à l'indice N provoque une erreur fatale de dépassement de capacité (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure de contrôle d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt minimales : (1) le succès (élément trouvé), et (2) l'échec (épuisement complet de la structure sans succès).